

ANTHROPIC · ENGINEERING 实践

Claude 计算机与浏览器使用 最佳实践

从分辨率到防护，构建可靠的代理体验

本次内容 / Agenda

1 分辨率与坐标

预降采样、API 上限、坐标对齐公式

2 诊断点击问题

偏移、近失、错点的根因排查

3 模型与思考努力

Sonnet · Opus · Haiku 的取舍

4 Prompt Injection 防御

训练、分类器、人工三层防线

5 长会话上下文管理

缓存断点 · 滚动缓冲 · 压缩

6 教 Claude 完成任务

演示模式与实验功能

PART 01

分辨率与坐标对齐

单一影响力最大的优化

最关键的一条：预降采样

在调用 API 之前，把截图压到模型的视觉上限以内。

这是文章中明确点名的**单一影响力最大的优化**。

- 避免后端"静默降采样"导致的坐标错位
- 显著减少 token 用量与延迟
- 让模型看到与你期望一致的像素结构

⚠ 常见陷阱

把原始 4K 截图直接送进模型 → 内部被压缩 → 模型返回的坐标无法映射回你的屏幕。

点击始终偏左、偏上，往往就是这个原因。

API 视觉输入 · 上限速查

Claude Sonnet 4.6

1568

最长边 px

1.15

megapixels

推荐起点 1280 × 720

Claude Opus 4.7

2576

最长边 px

3.75

megapixels

推荐起点 1920 × 1080

超出上限的截图会被**静默**压缩 — 坐标对齐随之失效

消息顺序：文本在前，图像在后

× 反模式

```
messages = [{  
  "role": "user",  
  "content": [  
    {"type": "image", ...},  
    {"type": "text",  
      "text": "请点击登录按钮"}  
  ]  
}]
```

模型先读图，再读目标。注意力分配不利。

✓ 推荐

```
messages = [{  
  "role": "user",  
  "content": [  
    {"type": "text",  
      "text": "请点击登录按钮"},  
    {"type": "image", ...}  
  ]  
}]
```

先建立目标，再扫描截图 — 点击准确率提升。

坐标缩放：把 API 坐标映射回真实屏幕

截图被压缩后，模型返回的是压缩坐标空间。在真正驱动鼠标前必须线性缩放回原始分辨率。

```
screen_x = int(api_x * (screen_w / display_w))
screen_y = int(api_y * (screen_h / display_h))
```

高级做法：使用 "**max API fit**" 算法，在不超出上限的前提下保留宽高比。

🎯 校验清单

- 实际截图尺寸 = 你声明的 `display_w / h`
- 缩放方向与轴一致（不要 swap x / y）
- HiDPI 屏幕的设备像素比要单独乘入
- 多显示器时使用窗口本地坐标

PART 02

诊断点击问题

偏移、近失、错点 — 三类症状对应三种根因

点击失败的 三类症状

症状	最可能的根因	解决方案
恒定偏移 每次都偏同一方向	声明的显示尺寸与实际截图不一致	核对 <code>display_w / h</code> ，确认坐标缩放因子
近失 擦边而过	目标元素过小，受降采样模糊	启用 zoom、放大目标、或改用键盘导航
点错元素 点到旁边相似项	提示语义模糊，缺少位置上下文	在 prompt 中补充"左侧/第一行/标题旁"

小目标的 四种应对策略

启用 Zoom

对密集 UI 增加局部缩放工具，让模型先聚焦再点击。

放大目标

如果是你可控的界面，直接把按钮、链接、可点击区域做得更大。

键盘导航

Tab / Enter / 快捷键比像素级点击更可靠，尤其在表单与菜单中。

控制源图分辨率

避免对已经被压缩过的源图再次降采样，叠加压缩会让小目标彻底糊掉。

PART 03

选择模型与思考努力

精度、推理、延迟，三方权衡

模型选择 · 三种取舍

Sonnet 4.6

机械精度更强

- 对重度降采样最鲁棒
- 高吞吐的代理首选
- 性价比平衡

Opus 4.7

推理更深

- 点击精度与 Sonnet 相当
- 复杂规划与异常处理
- 支持更高 megapixel 上限

Haiku 4.5

延迟最低

- 对延迟敏感的任务
- 简单、可预测的工作流
- 大规模批处理

思考努力：用一半的 token 换接近峰值的表现

Opus 4.7

- **high** 默认推荐 — 半数 token 达到近峰值表现
- **low** 成本敏感场景
- **max** 复杂的一次性任务

Sonnet 4.6 系列

- **medium** 性价比最佳
- **low** 高吞吐工作负载
- **off** 简单且定义良好的工作流

原则：先把分辨率与提示词调对，再用思考努力做最后的精度提升。

PART 04

Prompt Injection 防御

把所有网页内容视为不可信

三层防御·纵深策略

1

训练时鲁棒性

模型在训练阶段就被强化抵抗注入文本

2

实时分类器

官方 `computer_20251124` 工具自动扫描可疑指令

3

红队评估

持续的对抗性测试，发现新攻击面



工程侧实践

- 高风险动作要求 **human-in-the-loop**
- 严格收敛代理权限（最小授权）
- 对所有动作做审计日志
- 把网页内容视作**不可信**输入

PART 05

长会话的上下文管理

让代理跑得久，而且跑得起

三层叠加 · 缓存 → 滚动 → 压缩

📌 缓存断点

最多 3 个 ephemeral cache breakpoint, 放在最近的 tool 结果上。

让常用前缀命中缓存, 显著降低成本。

🔄 滚动缓冲

仅保留最近 `keep_n` 张截图, 老的批量裁掉。

默认 `keep_n=3`, `interval=25`

⚡ 压缩

token 超阈值 (约 150k) 时, 把会话压成摘要再继续。

保留任务目标、关键发现、待办事项。

三层效果**复利叠加**: 单独用任意一层都不如组合起来。

PART 06

提升可靠性 · 教会 Claude

用演示而非描述，传递复杂 workflow

演示驱动：Show, Don't Tell

用户亲自完成一次任务，系统记录全过程；Claude 在回放时把它当作**参考脚本**，并根据 UI 变化自适应。

数据模型记录的要素：

- 动作类型（点击 / 输入 / 导航 / 滚动）
- 人类可读的步骤描述
- 选择器与坐标
- 带视觉标记的截图
- 可选：语音解说转写

Strict 严格

逐步严格复现，几乎不偏离录制路径。

Adaptive 自适应 推荐默认

把演示当指引，按当前 UI 调整。

Goal-oriented 目标导向

只关心结果，自主选择路径。

实验性功能 · 两把利器

Batch Tools

把多个连续动作合并到一次调用，**显著降低延迟**。

 不要在依赖视觉反馈的步骤上使用 — 没看到第 N 步结果就发出第 N+1 步，错误会级联。

Advisor Tool

把"执行模型 + 更高智能的顾问模型"配对：执行端遇到困境时，可调用 Advisor 请求策略建议。

用**便宜的模型跑路径**，把贵的模型留给关键决策。

六条 关键点

01 分辨率决定准确率
预降采样是单一影响力最大的优化

02 文本先于图像
消息顺序直接影响点击命中率

03 medium / high 是甜点
思考努力不必拉满，性价比最优

04 上下文管理三层叠加
缓存断点 + 滚动缓冲 + 压缩

05 安全靠多层防御
训练 + 分类器 + 人工把关

06 演示比描述更可靠
复杂工作流用 Show, Don't Tell

END · 谢谢

从分辨率到信任 每一层都值得打磨

claude.com/blog/best-practices-for-computer-and-browser-use-with-claude